
SUBGOALCALC: Certified Decomposition for Lean Theorem-Proving Agents

Anonymous Author(s)

Affiliation

Address

email

Abstract

Subgoals now drive theorem-proving agents across extraction, search, scheduling, and reuse. We present SUBGOALCALC, a certified decomposition layer for Lean 4 that turns subgoals into proof obligations carrying witnesses, dependencies, and replay semantics. Its central object is a certified decomposition: a root goal, residual obligations, an acyclic dependency relation, and a Lean-checked assembly witness reconstructing the root theorem from residual proofs. The layer provides four exact operators: normalization, duplicate collapse, overlay, and refinement; together with dependency-aware schedule selection and replay-backed lemma lifting.

Across exact suites, the operator basis reduces 1,000 certified flat decompositions to four structural cores; a dependency suite contributes 750 certified DAGs with depth up to 27; and 1,500 certified transforms change execution mode exactly at crossings of the dependency-edge boundary. In HILBERT, kernel-faithful extraction raises end-to-end success from 0/6 to 6/6 over textual extraction and from 4/6 to 6/6 over a retry baseline, with 12/12 helper replays, 6/6 dependent-helper replays, and zero extraction-related recovery calls. On DEPBENCH, certified dependencies select staged execution exactly on the four dependency-bearing cases, raising contract satisfaction from 2/6 to 6/6 and reducing schedule cost from 54 to 14. Replay-backed helper lemmas compress chain plans, transfer across held-out HILBERT targets, and flip one DEPBENCH failure to success. The same control plane closes a white-box AESOP slice from 0/3 to 3/3 and produces exact certified decompositions for all 4,937 FORMALML examples, localizing the remaining replay boundary to provisioning, serializer effects, and source-aligned repair.

1 Introduction

Subgoal structure now appears throughout formal theorem proving. FORMALML isolates subgoal completion as a distinct benchmark task in LEAN 4 [7]; Zhao et al. and SubgoalXL use subgoal-based proofs to improve in-context proving and expert learning in Isabelle [8, 9]; HILBERT uses recursive decomposition to connect informal reasoning with formal proof construction [6]; DeepSeek-Prover-V2 uses subgoal decomposition to build synthetic training data and stabilize reinforcement learning [5]; Bourbaki treats self-generated subgoals as dynamic search targets [10]; and MATHLIBLEMMA shows that many missing intermediate facts are natural candidates for library growth [4]. Across these systems, subgoals organize prover work by structuring generation, search, training, and reuse around intermediate obligations.

The corresponding representations usually materialize subgoals as proof sketches, helper-theorem strings, or search states. These forms can drive generation and search, but they do not by themselves record the operational structure needed to audit and reuse a lifted obligation. A prover must be able to check whether an extracted helper theorem still expresses the same obligation after being

removed from its source proof, determine which helpers are independent and which depend on earlier discharged obligations, and replay a local helper theorem against the original proof step when that helper is reused.

We present SUBGOALCALC, a certified decomposition layer for LEAN 4. Its central object is a *certified decomposition*: a root goal, a finite family of residual obligations, an operational dependency relation among those obligations, and an assembly witness showing that proofs of the residual obligations reconstruct a proof of the root theorem. The decomposition is therefore represented as a proof-theoretic object inside LEAN 4, with explicit obligations, dependencies, and reconstruction evidence.

We use the term *control plane* for the representation layer whose outputs are consumed by downstream proving components when they choose schedules, lift helper theorems, or reinject lemmas, while preserving replay links back to LEAN 4. Certified decompositions support exact comparison and transformation through four operators: normalization, duplicate collapse, overlay, and refinement. Each operator rebuilds the assembly witness and the resulting object is checked again in LEAN 4, giving a compositional algebra over local proof obligations.

The dependency relation carried by a decomposition gives the controller enough structure to select admissible schedules. Independent obligations can be exposed to parallel solving, while dependency-bearing obligations impose staged execution. In our downstream experiments, this dependency geometry changes the internal behavior of a real proving stack and determines which schedules satisfy the induced contract.

The same representation also supports replay-backed lemma reuse. Residual obligations can be lifted to closed theorems, minimized, replay-checked against the original local proof body, and packaged for later use. This turns local proof obligations into small helper libraries with explicit replay evidence, complementing offline theorem-mining approaches.

We evaluate SUBGOALCALC across generated exact suites, downstream prover integrations, and corpus-scale extraction. The exact suites test the algebra on dependency-bearing DAGs and geometry-changing transforms. In an integration with HILBERT, certified extraction removes structural extraction failures on a curated stress slice. On DEPBENCH, the controller satisfies a schedule contract induced by dependency geometry. Replay-backed helper lemmas compress proof plans, transfer across held-out HILBERT targets, and cross from HILBERT into DEPBENCH. A second downstream study closes a small white-box AESOP feasibility slice. On FORMALML, SUBGOALCALC performs exact goalframe extraction on all 4,937 examples and localizes the remaining replay boundary through audited probes.

Overall, SUBGOALCALC provides a certified representation of local proof decomposition in LEAN 4, an exact algebra over that representation, a dependency-aware controller for schedule selection, a replay-backed lemma pipeline for proof-plan compression and transfer, and a corpus-scale account of exact extraction and replay opening on FORMALML.

2 Certified decomposition

2.1 Goal frames and certified decompositions

A local helper theorem inside a LEAN 4 proof carries more structure than its target formula. It has a local context, a dependency profile, and elaboration state. SUBGOALCALC represents this information through a *goal frame*.

Definition 1 (Goal frame). A *goal frame* is a tuple

$$g = (\Gamma, T, \Theta, \Delta, s),$$

where Γ is the ordered local context, T is the target type, Θ is the closed theorem type obtained by abstracting Γ over T , Δ records dependencies on earlier local helper theorems, and s records whether unresolved synthetic metavariables remain.

In the implementation, a goal frame is computed from a goal metavariable in LEAN 4’s metaprogramming layer. Expressions are sanitized before comparison: assigned metavariables are instantiated, metadata and annotation wrappers are removed, and reducible definitions are weak-head normalized. This produces stable closed theorem types for equality tests, replay, and witness typing. Universe

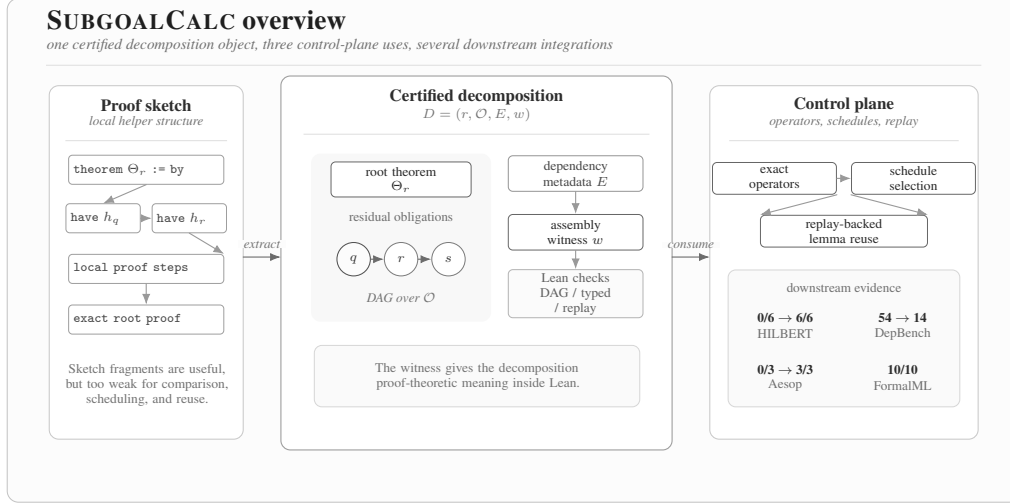


Figure 1: Overview of SUBGOALCALC. A single certified decomposition object is used for exact transformation, schedule selection, replay-backed lemma extraction, and downstream integration.

parameters and instance binders remain part of the elaborated theorem type. Unresolved synthetic metavariables cause certification rejection.

Definition 2 (Certified decomposition). *Let r be a root goal frame. A certified decomposition of r is a tuple*

$$D = (r, O, E, w),$$

where $O = (o_1, \dots, o_m)$ is an ordered family of residual obligations, $E \subseteq O \times O$ is an acyclic dependency relation, and

$$w : \Theta_{o_1} \rightarrow \dots \rightarrow \Theta_{o_m} \rightarrow \Theta_r$$

is an assembly witness accepted by LEAN 4.

The witness gives the decomposition a direct proof-theoretic meaning: proofs of the residual obligations reconstruct a proof of the root theorem.

The witness and the dependency relation have different roles. The witness is checked directly by LEAN 4. The dependency relation records dependencies exposed by elaborated helper bodies, is checked for acyclicity, and is used operationally through replay and schedule evaluation. In this paper, E is the extracted operational dependency structure associated with certification and replay.

Proposition 1 (Certification soundness). *If $D = (r, O, E, w)$ is certified and each Θ_{o_i} is inhabited, then Θ_r is inhabited.*

Complete proofs appear in the appendix.

2.2 Exact operators

SUBGOALCALC equips certified decompositions with four operators.

Normalization. Normalization canonicalizes obligation order and rewrites dependency references accordingly. It preserves certification and the root theorem.

Duplicate collapse. Duplicate collapse quotients obligations by equality of sanitized closed theorem type, redirects all incoming references to a representative obligation, and rebuilds the witness on the quotient. This quotient is sound at the proof-obligation layer. Source provenance is stored separately during replay-backed reuse, keeping theorem-key equality and proof origin as distinct pieces of evidence.

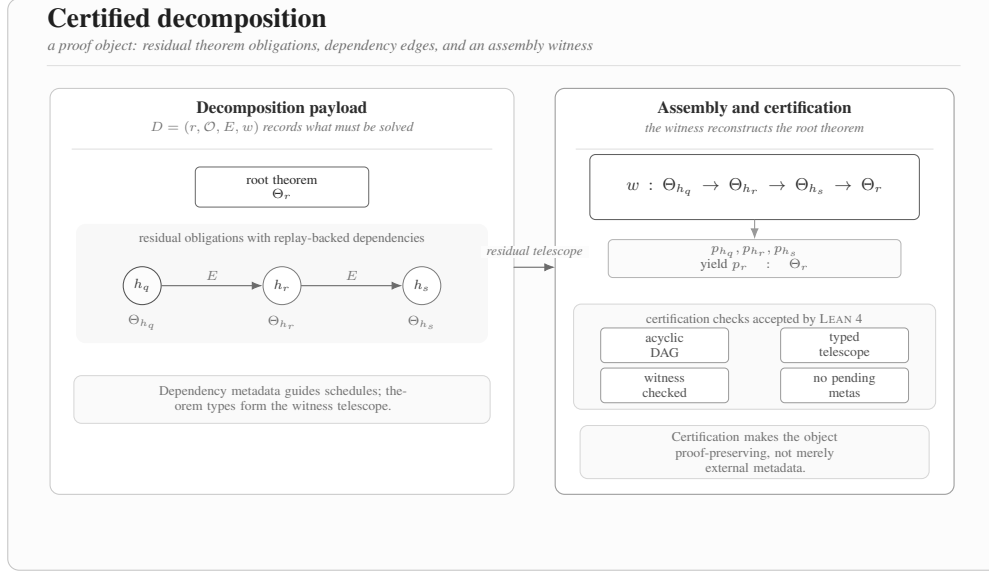


Figure 2: Certified decomposition as a proof object. Residual obligations carry dependency edges and an explicit assembly witness accepted by LEAN 4.

Overlay. Given two certified decompositions with the same root theorem, overlay forms their certified superposition. It unions the obligation families after identifier shifting and weakens the witness over the larger telescope. Overlay acts as a join in decomposition space. In this paper, it supports exact comparison and aggregation over certified decompositions.

Refinement. Refinement substitutes a certified child decomposition for one selected residual obligation in a parent decomposition. If the child root theorem matches that residual theorem, the refined witness is obtained by functional composition.

Proposition 2 (Certification preservation). *Normalization, duplicate collapse, overlay, and refinement preserve certification.*

These operators act on certified objects whose outputs are re-checked by LEAN 4. The implementation also computes a reduced *structural core* by canonicalization and quotienting, which is used for exact equality and algebraic counting in the experiments.

2.3 Schedules induced by dependency structure

A decomposition induces a scheduling problem. Independent residual obligations can be solved in parallel, while dependency-bearing obligations require staged execution.

Definition 3 (Admissible schedule). *Let $D = (r, \mathcal{O}, E, w)$ be a certified decomposition. An ordered partition $(B_0, \dots, B_k)$ of $\mathcal{O}$ is admissible if for every edge $(o_i, o_j) \in E$, the batch containing o_i appears strictly before the batch containing o_j .*

Operationally, the schedules are *flat parallel*, which places all obligations in one batch, and *dependency staged*, which groups obligations by depth in the dependency DAG.

Theorem 1 (Schedule laws). *For every certified decomposition: (i) the depth-layered schedule is admissible; and (ii) the flat-parallel schedule is admissible if and only if the dependency graph is edgeless.*

These laws turn dependency structure into a controller decision used in the experiments as both a schedule contract and a regression-test target for dependency-aware execution.

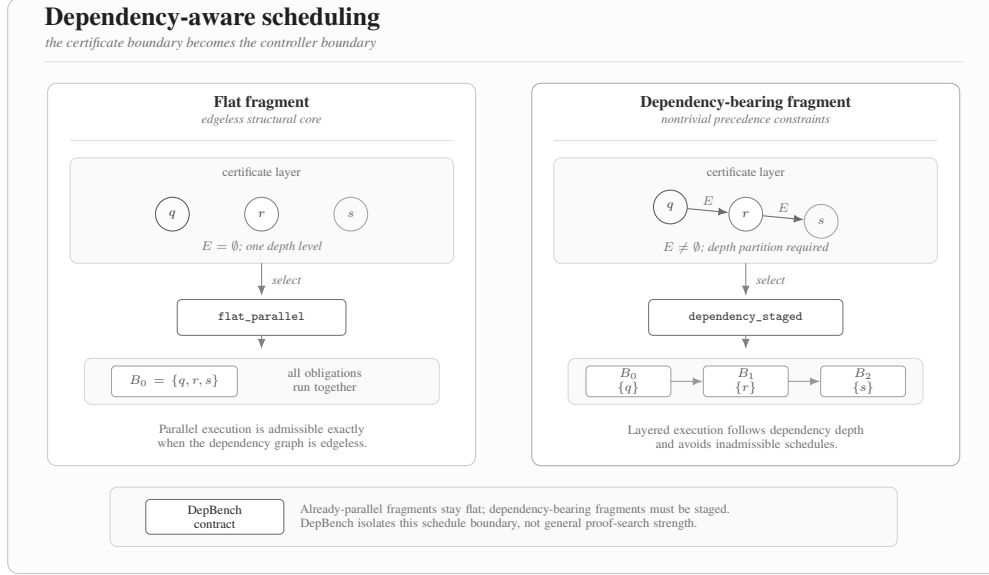


Figure 3: Dependency-aware schedule selection. The same schedule boundary appears in the certificate layer and in the controller’s choice between parallel and layered execution.

137 3 Control and replay-backed reuse

138 3.1 Kernel-faithful extraction from proof sketches

139 The preferred extraction route operates on elaborated proof structure. Given a proof sketch with local
 140 helper theorems, SUBGOALCALC builds goal frames for the root and residual obligations, constructs
 141 the dependency DAG, and emits a certified decomposition whose witness is checked by LEAN 4. This
 142 preserves local structure that textual extraction can drop, especially when helper theorems depend on
 143 earlier local steps.

144 3.2 Controller

145 The controller consumes a certified decomposition and selects a solving schedule. In the experiments,
 146 we use four modes: off, flat_parallel, dependency_staged, and auto. The auto mode
 147 applies the schedule laws from Section 2: edgeless decompositions are assigned to flat-parallel
 148 execution, and dependency-bearing decompositions are assigned to dependency-staged execution.
 149 The controller also accounts for helper-lemma reinjection, since successful reinjection can collapse
 150 dependency edges and move a target from the staged fragment to the flat fragment.

151 3.3 Replay-backed lemma lifting

152 Residual obligations are turned into reusable helper lemmas through lifting, replay, and reinjection.

153 **Lifting.** A residual obligation is lifted to a closed theorem statement by abstracting its local context
 154 and retaining the declarations needed for a well-typed theorem.

155 **Replay.** The lifted statement is paired with the original local proof body, and the resulting theorem
 156 is replay-checked by LEAN 4. Replay establishes that the lifted theorem still corresponds to the local
 157 proof step after abstraction.

158 **Reinjection.** Replay-checked helper lemmas are packaged into small libraries and reintroduced
 159 into later proof attempts. Reinjection is selective. A transferred lemma is useful when it matches a
 160 residual obligation in the target proof and changes the plan geometry in a favorable way, for example
 161 by eliminating a dependency-bearing helper and enabling a flat-parallel schedule.

162 The pipeline observes lemma reuse through downstream behavior: a helper lemma survives lifting
163 and replay, is reintroduced into a later proof attempt, and is credited when its presence changes the
164 selected plan or the proof search outcome.

165 4 Experimental design

166 The evaluation follows the lifecycle of a certified decomposition: construction and transformation,
167 use by a downstream controller, and reuse through replay-checked helper lemmas.

168 **Exact suites.** Generated certified suites validate the implementation and the algebraic operators.
169 The first suite contains 1,000 flat decompositions generated from four operator families. The second
170 contains 750 certified dependency-bearing DAGs across three families, with up to 28 obligations and
171 dependency depth up to 27. A third suite contains 1,500 certified transforms that cross the schedule
172 boundary in both directions.

173 **HILBERT integration.** We use a curated six-problem slice that stresses local helper-theorem
174 extraction and dependency handling. The main comparison uses a textual extraction baseline. We
175 also report a retry-enabled baseline.

176 **DEPBENCH.** DEPBENCH is a dependency-sensitive scheduling benchmark with six cases: two
177 flat-parallel cases and four dependency-bearing cases. It isolates controller behavior while keeping
178 the rest of the proving stack fixed, and serves as a constructed schedule-contract benchmark.

179 **Replay-backed reuse.** We study reuse under within-problem reinjection on the chain slice of the
180 HILBERT benchmark, held-out transfer across six HILBERT targets, and cross-corpus transfer
181 from HILBERT into DEPBENCH and FORMALML.

182 **White-box augmentation.** We evaluate a second downstream regime on a small AESOP slice with
183 three flat-fragment fixtures. One fixture contains duplicate residual obligations and exposes quotient
184 savings directly. This slice serves as a white-box feasibility check.

185 **FORMALML.** We use the full FORMALML train split of 4,937 examples to study corpus-scale
186 extraction and replay. The corpus produces 3,041 residual obligations, 2,362 unique theorem keys,
187 and 679 duplicate keys. For replay, we report exact extraction at scale and use stratified probes to
188 separate provisioning, serializer normalization, and source-aligned family repair.

189 **Metrics.** The main metrics are certification rate, replay rate, dependency recall, controller contract
190 satisfaction, admissibility, schedule cost, proof-plan cost under reinjection, transfer selectivity, and
191 exact corpus coverage. Schedule and proof-plan costs are structural accounting scores, so we report
192 their components alongside the totals.

193 5 Results

194 5.1 The exact algebra exposes a small stable basis and a dependency-bearing regime

195 The large flat suite reduces 1,000 certified decompositions to four structural cores. All 1,000
196 entries satisfy the schedule laws. Since the suite is edgeless, flat-parallel execution is admissible
197 throughout. The dependency-bearing suite exposes a separate regime: 750 certified DAGs, 500
198 of them dependency-bearing, with 10,750 total dependency edges and depth up to 27. Across
199 1,500 certified transforms, 750 cases cross the schedule boundary. These suites provide large-scale
200 regression coverage for exact-layer invariants.

201 The algebra preserves certification and exercises the same schedule boundary used later by the
202 controller. The appendix reports the exact counts, while the main text focuses on the structural
203 regimes exposed by the transform layer.

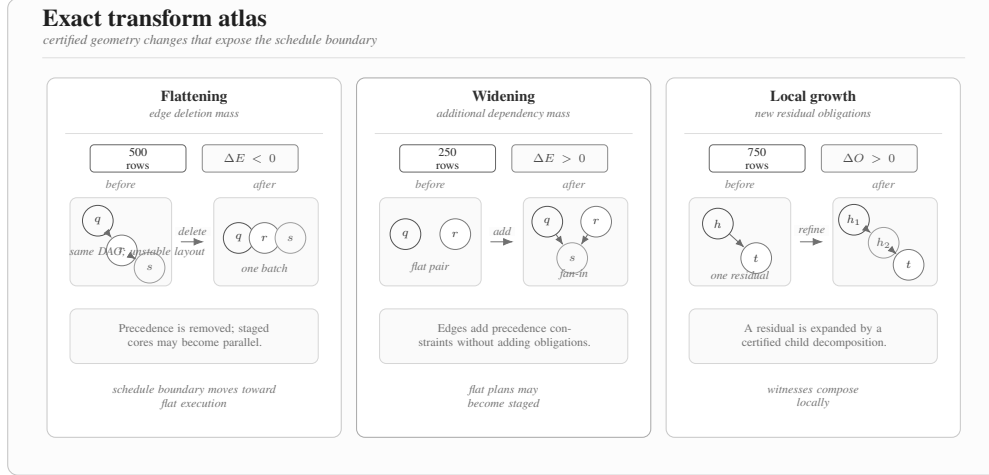


Figure 4: Exact transform atlas. The certified transform layer already exposes flattening, widening, local growth, and schedule-boundary crossings.

Table 1: HILBERT integration.

Metric	Textual baseline	Retry baseline	SUBGOALCALC	Best delta
Final successes	0/6	4/6	6/6	+6, +2
Helper replay rate	7/12	—	12/12	+5
Dependent-helper replay	1/6	—	6/6	+5
Invalid subgoal checks	12	0	0	−12
Extraction-related reasoner calls	14	4	0	−14, −4
Dependency lift rate	0.000	0.417	0.417	+0.417

5.2 Kernel-faithful extraction removes structural failures in HILBERT

On the curated six-problem HILBERT stress slice, the textual extraction route compiles all six sketches but replays only 7 of 12 helper theorems and only 1 of the 6 dependency-bearing helpers. The certified route replays all 12 helpers and all 6 dependent ones, with perfect dependency-edge recovery.

The same difference appears in end-to-end behavior. Against the textual baseline, success rises from 0/6 to 6/6, invalid helper checks drop from 12 to 0, and extraction-related reasoner calls fall from 14 to 0. Against the retry-enabled baseline, success rises from 4/6 to 6/6 and extraction-related reasoner calls fall from 4 to 0.

5.3 Controller decisions become exact on dependency-bearing cases

On DEP BENCH, the controller is the only changing component. The baseline keeps all four dependency-bearing cases in flat parallel, violating the induced schedule constraints. The certified controller switches exactly those four cases to dependency-staged execution, leaves the already-flat cases unchanged, and raises controller contract satisfaction from 2/6 to 6/6. This experiment isolates the schedule contract induced by dependency geometry.

Schedule-cost gains concentrate in the gated dependency-bearing slice, while the already-admissible parallel slice is unchanged. Generate-proof calls stay fixed, inadmissibility risk drops from 4 to 0, and helper reuse credit rises from 0 to 4. This matches the exact transform results: the controller intervenes on decompositions located on the dependency-bearing side of the schedule boundary.

5.4 Replay-backed helper lemmas compress proof plans and transfer selectively

Within-problem reinjection on the chain slice of the curated HILBERT benchmark collapses both chain cases from layered plans to one-batch plans and reduces proof-plan cost from 16 to 6. In

Table 2: Dependency-aware controller on DEP BENCH.

Metric	Baseline	SUBGOALCALC controller	Delta
Contract-satisfying runs	2/6	6/6	+4/6
Gated dep.-bearing runs satisfied	0/4	4/4	+4/4
Dependency-staged runs	0	4	+4
Inadmissible schedule runs	4	0	-4
Total schedule cost	54	14	-40

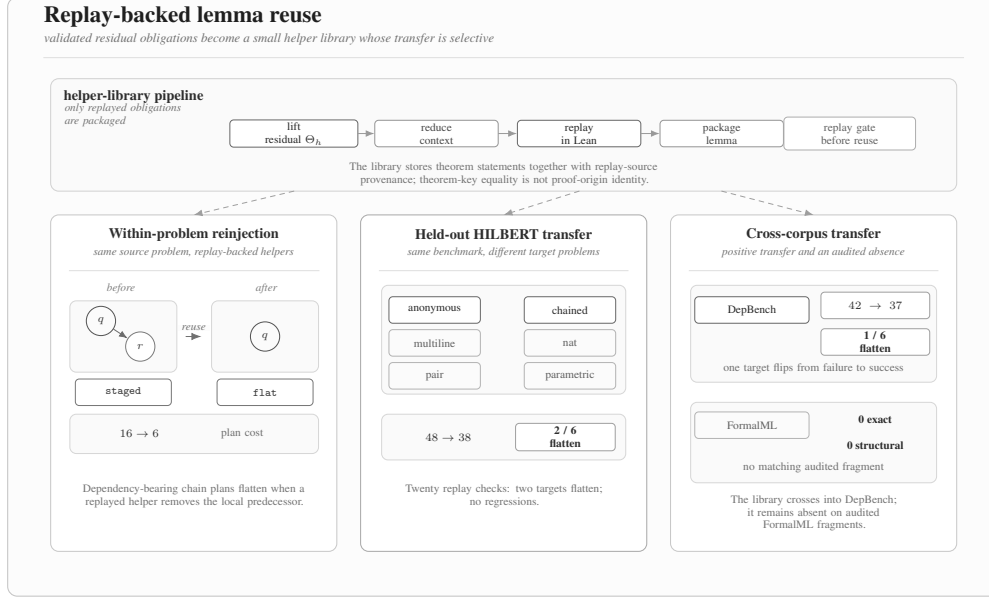


Figure 5: Replay-backed lemma reuse and transfer. It compresses dependency-bearing plans locally, transfers across held-out HILBERT targets, and crosses into DepBench while remaining absent on the audited FormalML fragments.

226 held-out transfer, a four-entry replay-backed micro-library spanning two structural families produces
 227 20 replay-checked placements across six distinct HILBERT targets: two targets flatten, four remain
 228 unchanged, none is harmed, and total proof-plan cost falls from 48 to 38.

229 The same library transfers selectively across corpora. In DEP BENCH, one target flattens and flips
 230 from failure to success; on the audited FORMALML fragments, exact overlap remains zero because
 231 the library replays but its theorem keys do not match the audited residual obligations. The appendix
 232 reports the detailed transfer table, while the main text uses these results to characterize replay-backed
 233 reuse as fragment-selective compression of dependency-bearing plans.

234 5.5 A second downstream regime closes on a white-box AESOP slice

235 The same representation supports a flat, white-box proof-search regime. On three AESOP fixtures, all
 236 in the flat fragment, local forward rules derived from the decomposition close the full slice from 0/3
 237 to 3/3. One fixture contains duplicate residual obligations, and quotienting removes one redundant
 238 obligation before rule generation. This slice serves as a white-box feasibility check for rule generation
 239 from certified decompositions.

240 5.6 FORMALML separates exact extraction from replay opening

241 At corpus scale, FORMALML behaves differently from the dependency-sensitive slices. The extrac-
242 tion layer is exact on all 4,937 examples and all 3,041 residual obligations. The resulting residual
243 graph is predominantly flat, so FORMALML primarily evaluates exact extraction and replay opening.

244 Replay coverage depends on the available workspace roots and the normalization layer used during
245 checking. In the main workspace, missing external project roots block replay; once those roots are
246 provisioned at the pinned revisions, the replay boundary opens on all audited probes. On the stratified
247 probes, raw exact replay succeeds on 6/10, 3/10, and 3/10 examples for the 100, 500, and full slices.
248 Serializer normalization raises the two larger probes to 5/10. Source-aligned family repair closes the
249 remaining residual rows, producing 10/10 final replay on all three probes. These repair families are
250 reported as a diagnostic replay-opening layer.

251 6 Related work

252 Recent work has established several roles for subgoals in formal reasoning. FORMALML formulates
253 subgoal completion as a benchmark task in LEAN 4 and studies local obligation discharge under com-
254 plex contexts and retrieval [7]. Zhao et al. and SubgoalXL use subgoal-based proofs as supervision
255 targets for in-context proving and expert learning in Isabelle [8, 9]. HILBERT uses recursive decom-
256 position to connect informal reasoning with formal proof construction, while DeepSeek-Prover-V2
257 uses decomposition to synthesize training data and stabilize reinforcement learning [5, 6]. Bour-
258 baki formalizes self-generated subgoals as dynamic control targets in goal-conditioned search [10].
259 MATHLIBLEMMA studies theorem-library growth by mining missing folklore lemmas [4].

260 SUBGOALCALC sits between extraction, search, and reuse by providing a certified decomposition
261 object that records residual obligations, dependency geometry, and an assembly witness accepted by
262 LEAN 4. Existing decomposition-based systems use subgoals as task units, prompts, search states,
263 or training artifacts; SUBGOALCALC gives the decomposition itself replay semantics, algebraic
264 operations, and controller-facing structure. The resulting object is close to proof planning [1] and
265 white-box proof search in AESOP [3], with an implementation in modern LEAN 4 and an interface
266 aimed at current decomposition-based theorem-proving agents.

267 7 Conclusion and limitations

268 SUBGOALCALC represents local proof decomposition as a certified object inside LEAN 4, containing
269 residual obligations, dependency geometry, and an assembly witness checked by the prover. This
270 representation gives exact transformation, dependency-aware scheduling, and replay-backed helper-
271 lemma reuse a common control plane.

272 Across the evaluated regimes, the same control plane supports algebraic validation, extraction,
273 scheduling, proof-search integration, and corpus-scale replay analysis. The exact suites validate the
274 algebra and exercise the schedule boundary at scale. The HILBERT integration shows that kernel-
275 faithful extraction removes structural failures that remain under textual extraction. The DEPBENCH
276 study isolates schedule selection and shows that dependency geometry can drive exact controller
277 decisions. The AESOP slice shows that the same representation can feed a second white-box proof-
278 search regime. The FORMALML study shows corpus-wide exact extraction and separates extraction
279 from replay opening on audited probes.

280 The assembly witness is checked directly by LEAN 4, while the dependency relation is operational
281 metadata extracted from elaborated helper bodies and validated through acyclicity, replay, and
282 schedule behavior. The exact suites are generated validation batteries. The HILBERT, DEPBENCH,
283 and AESOP studies are small, purpose-built stress slices. The FORMALML replay study reports
284 stratified probe results, with source-aligned family repair used as a diagnostic replay-opening layer.

285 These boundaries point to the next technical target: expanding the reusable fragment while preserving
286 replay fidelity. The results establish certified decomposition as a practical representation layer for
287 theorem-proving agents, connecting local proof structure to exact algebra, controller decisions, and
288 reusable helper libraries.

References

- [1] Alan Bundy. Proof planning. In *Proceedings of the Third International Conference on Artificial Intelligence Planning Systems*, AIPS 1996, pages 261–267. AAAI Press, 1996. URL <https://cdn.aaai.org/AIPS/1996/AIPS96-033.pdf>.
- [2] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In André Platzer and Geoff Sutcliffe, editors, *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635, Cham, 2021. Springer. doi: 10.1007/978-3-030-79876-5_37. URL https://doi.org/10.1007/978-3-030-79876-5_37.
- [3] Jannis Limperg and Asta Halkjær From. Aesop: White-box best-first proof search for Lean. In *Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs*, CPP 2023, pages 253–266, New York, NY, USA, 2023. Association for Computing Machinery. doi: 10.1145/3573105.3575671. URL <https://doi.org/10.1145/3573105.3575671>.
- [4] Xinyu Liu, Zixuan Xie, Amir Moeini, Claire Chen, Shuze Daniel Liu, Yu Meng, Aidong Zhang, and Shangtong Zhang. MathlibLemma: Folklore lemma generation and benchmark for formal mathematics. *arXiv preprint arXiv:2602.02561*, 2026. doi: 10.48550/arXiv.2602.02561. URL <https://arxiv.org/abs/2602.02561>.
- [5] Z. Z. Ren, Zhihong Shao, Junxiao Song, Huajian Xin, Haocheng Wang, Wanjia Zhao, Liyue Zhang, Zhe Fu, Qihao Zhu, Dejian Yang, Z. F. Wu, Zhibin Gou, Shirong Ma, Hongxuan Tang, Yuxuan Liu, Wenjun Gao, Daya Guo, and Chong Ruan. DeepSeek-Prover-V2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition. *arXiv preprint arXiv:2504.21801*, 2025. doi: 10.48550/arXiv.2504.21801. URL <https://arxiv.org/abs/2504.21801>.
- [6] Sumanth Varambally, Thomas Voice, Yanchao Sun, Zhifeng Chen, Rose Yu, and Ke Ye. Hilbert: Recursively building formal proofs with informal reasoning. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=GN80dkTo3B>.
- [7] Xiao-Wen Yang, Zihao Zhang, Jianuo Cao, Zhi Zhou, Zenan Li, Lan-Zhe Guo, Yuan Yao, Taolue Chen, Yu-Feng Li, and Xiaoxing Ma. FormalML: A benchmark for evaluating formal subgoal completion in machine learning theory. *arXiv preprint arXiv:2510.02335*, 2025. doi: 10.48550/arXiv.2510.02335. URL <https://arxiv.org/abs/2510.02335>.
- [8] Xueliang Zhao, Wenda Li, and Lingpeng Kong. Subgoal-based demonstration learning for formal theorem proving. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 60832–60865. PMLR, 2024. URL <https://proceedings.mlr.press/v235/zhao24h.html>.
- [9] Xueliang Zhao, Lin Zheng, Haige Bo, Changran Hu, Urmish Thakker, and Lingpeng Kong. SubgoalXL: Subgoal-based expert learning for theorem proving. *arXiv preprint arXiv:2408.11172*, 2024. doi: 10.48550/arXiv.2408.11172. URL <https://arxiv.org/abs/2408.11172>.
- [10] Matthieu Zimmer, Xiaotong Ji, Rasul Tutunov, Anthony Bordg, Jun Wang, and Haitham Bou Ammar. Bourbaki: Self-generated and goal-conditioned MDPs for theorem proving. *arXiv preprint arXiv:2507.02726*, 2025. doi: 10.48550/arXiv.2507.02726. URL <https://arxiv.org/abs/2507.02726>.

A Formal properties

This appendix gives complete proofs of the structural results used in the main text.

Proposition 3 (Certification soundness). *If $D = (r, O, E, w)$ is certified and each residual theorem Θ_{o_i} is inhabited, then the root theorem Θ_r is inhabited.*

336 *Proof.* For each residual obligation o_i , let $p_i : \Theta_{o_i}$ be a proof term. Since the witness is well typed,

$$w p_1 \cdots p_m$$

337 is a term of type Θ_r . Hence Θ_r is inhabited. $\square$

338 **Proposition 4** (Normalization preserves certification). *If D is certified, then $\text{Normalize}(D)$ is*
 339 *certified and has the same root theorem.*

340 *Proof.* Normalization only permutes the residual telescope and rewrites dependency references
 341 accordingly. Let the original telescope be

$$\Theta_{o_1} \rightarrow \cdots \rightarrow \Theta_{o_m} \rightarrow \Theta_r,$$

342 and let π be the permutation induced by normalization. The normalized witness is

$$w_{\text{norm}}(x_1, \dots, x_m) = w(x_{\pi^{-1}(1)}, \dots, x_{\pi^{-1}(m)}).$$

343 This term has the normalized telescope type, the codomain remains Θ_r , and the dependency graph
 344 stays acyclic because normalization relabels nodes but adds no edges. $\square$

345 **Proposition 5** (Duplicate collapse preserves certification). *If D is certified, then $\text{Quotient}(D)$ is*
 346 *certified and has no more residual obligations than D .*

347 *Proof.* Duplicate collapse groups obligations by equality of their sanitized closed theorem type
 348 and keeps one representative per class. All references to eliminated nodes are redirected to the
 349 representative. Since obligations in the same class have the same theorem statement, the witness can
 350 be rewritten by replacing each eliminated argument by the corresponding representative argument.
 351 Collapsing duplicate nodes in a DAG cannot create a nontrivial directed cycle; at worst it creates
 352 self-loops, which are removed. Therefore the quotient remains acyclic and certified, with weakly
 353 fewer obligations. $\square$

354 **Proposition 6** (Overlay preserves certification). *Let D_1 and D_2 be certified decompositions with the*
 355 *same root theorem. Then $\text{Overlay}(D_1, D_2)$ is certified.*

356 *Proof.* Overlay takes the disjoint union of the obligation families after identifier shifting and preserves
 357 the shared root theorem. Since no new cross-edges are introduced between the two input graphs, the
 358 output graph is acyclic. If

$$w_1 : \Theta_{a_1} \rightarrow \cdots \rightarrow \Theta_{a_m} \rightarrow \Theta_r$$

359 is the witness of D_1 , then a valid witness for the overlaid telescope

$$\Theta_{a_1} \rightarrow \cdots \rightarrow \Theta_{a_m} \rightarrow \Theta_{b_1} \rightarrow \cdots \rightarrow \Theta_{b_n} \rightarrow \Theta_r$$

360 is obtained by weakening:

$$w_{\text{ov}}(x_1, \dots, x_m, y_1, \dots, y_n) = w_1(x_1, \dots, x_m).$$

361 Thus the overlaid decomposition is certified. $\square$

362 **Proposition 7** (Refinement preserves certification). *Let D_P be a certified parent decomposition and*
 363 *D_C a certified child decomposition whose root theorem matches one selected residual obligation of*
 364 *D_P . Then $\text{Refine}(D_P, D_C)$ is certified and has the same root theorem as D_P .*

365 *Proof.* Let the selected parent residual obligation have theorem statement Θ_o . Because the child root
 366 theorem is also Θ_o , the child witness has type

$$w_C : \Theta_{c_1} \rightarrow \cdots \rightarrow \Theta_{c_k} \rightarrow \Theta_o.$$

367 If the parent witness has type

$$w_P : \Theta_{p_1} \rightarrow \cdots \rightarrow \Theta_o \rightarrow \cdots \rightarrow \Theta_{p_m} \rightarrow \Theta_r,$$

368 then the refined witness is the composition

$$w_{\text{ref}}(\vec{x}, \vec{y}, \vec{z}) = w_P(\vec{x}, w_C(\vec{y}), \vec{z}).$$

369 The refined graph replaces one node by an acyclic child graph and redirects edges through the child
 370 root; acyclicity is preserved. The codomain remains Θ_r , so certification is preserved. $\square$

371 **Definition 4** (Depth). *For a certified decomposition $D = (r, O, E, w)$ and obligation $o \in O$, the*
 372 *depth $d(o)$ is the length of the longest directed path ending at o .*

373 **Theorem 2** (Depth-layered schedules are admissible). *Let $B_t = \{o \in O : d(o) = t\}$. Then*
 374 *$(B_0, B_1, \dots, B_k)$ is an admissible schedule.*

375 *Proof.* Take any edge $(o_i, o_j) \in E$. Any path ending at o_i can be extended by this edge to obtain a
 376 longer path ending at o_j . Therefore $d(o_i) < d(o_j)$, so o_i appears in an earlier batch than o_j . This is
 377 exactly admissibility. $\square$

378 **Corollary 1** (Flat parallel admissibility). *The one-batch flat-parallel schedule is admissible if and*
 379 *only if the dependency graph is edgeless.*

380 *Proof.* If the graph has no edges, there are no precedence constraints and the one-batch schedule
 381 is admissible. If there exists an edge (o_i, o_j) , admissibility requires o_i to appear strictly before o_j ,
 382 which is impossible in a single batch. $\square$

383 B Additional experimental detail

384 **Certification scope.** The Lean-checked assembly witness certifies root reconstruction from residual
 385 obligations. Dependency edges are replay-backed operational dependencies extracted from elaborated
 386 proof bodies and checked for acyclicity and replay consistency. We do not claim that the extracted
 387 DAG is a minimal semantic dependency basis.

388 **Exact suites.** The flat suite contains four certified operator families and 1,000 total rows. The
 389 dependency-bearing suite contains three families and 750 certified rows, including 250 chain and
 390 250 mixed cases. The transform suite contains 1,500 certified transforms with 750 schedule-mode
 391 crossings.

392 **HILBERT extraction study.** The curated slice contains six problems. The exact route replays all
 393 12 extracted helper theorems and all 6 dependency-bearing helpers. The textual route replays 7/12
 394 helper theorems and 1/6 dependency-bearing helpers.

395 **Controller benchmark.** DEP BENCH contains six cases: four dependency-bearing and two already-
 396 flat cases. Contract satisfaction measures whether the selected schedule agrees with the certified
 397 dependency geometry. Schedule cost is the benchmark’s aggregate cost function over selected
 398 schedules.

399 **Lemma reuse.** The replay-backed micro-library used in the transfer experiments contains four
 400 entries spanning two structural families. Held-out transfer reports only off-diagonal placements.
 401 Cross-corpus transfer uses the same replay-checked library without additional tuning.

402 **FORMALML.** On the full train split, 4,937 examples yield 3,041 residual obligations. For replay,
 403 we report three stratified probes of size 10, corresponding to the 100-example, 500-example, and
 404 full-slice audits. The replay progression separates exact extraction, provisioned statement replay,
 405 serializer normalization, and source-aligned family repair.

406 **White-box augmentation.** The AESOP slice contains three fixtures, all in the flat fragment. Rule
 407 augmentation uses local forward rules synthesized from replay-backed residual obligations.

Table 3: Lean feature handling.

Feature	Treatment	Boundary / failure mode
Universe parameters and instance binders	Preserved in the elaborated theorem type and replayed verbatim	No erasure-based quotienting across distinct elaborated binders
Local <code>let</code> declarations	Lifted through the reconstructed local context in declaration order	Fails if replay would leave unresolved metavariables
Reducible definitions	Sanitized by weak-head normalization at the reducible layer only	Not a full semantic normalization procedure
Synthetic metavariables	Rejected at certification / replay time if unresolved	Extraction is kernel-faithful only for accepted replayed terms
Duplicate theorem keys	Closed theorem-key equality is used for obligation quotienting	Replay-backed reuse keeps provenance and replay class separate

Table 4: Additional exact-suite statistics.

Suite	Entries	Max obligations	Max width	Max depth/stages	Edge total / delta
Flat certified suite	1,000	4	4	0	0
Dependency DAG suite	750	28	28	27	10,750
Certified transforms	1,500	29	—	26	−7,250 net

Table 5: HILBERT stress-slice taxonomy.

Problem	Textual baseline	Retry	Certified	Main structural issue
anon. chain	fail	success	success	anonymous helper lifting
chained	fail	fail	success	predecessor binder missing in helper replay
multiline	fail	success	success	multiline dependency preservation
dep.				
nat chain	fail	success	success	dependency-bearing local theorem promotion
pair chain	fail	success	success	pair-structured helper interface preservation
parametric	fail	fail	success	lifted parameter binder required for replay

Table 6: Cost components reported in the main text.

Benchmark	Setting	Risk	H	D	Calls	Overhead	Reuse / reinj.	Total
DEPBENCH	controller off	4	—	—	14	0	0	54
DEPBENCH	controller auto	0	—	—	14	4	4	14
HILBERT reinjection	without pack	—	4	2	6	2	2	16
HILBERT reinjection	with pack	—	2	0	4	0	0	6

Table 7: Replay-backed reinjection and transfer detail.

Setting	Tasks	Baseline cost	With library	Delta	Main effect
Chain slice (within-problem)	2	16	6	−10	both chain cases flatten
Held-out HILBERT transfer	6	48	38	−10	2 flattened targets, no regressions
Cross-corpus transfer to DEPBENCH	6	42	37	−5	one failure-to-success flip
Cross-corpus transfer to FORMALML	audited probes	0 overlap	0 overlap	—	fragment mismatch, not replay failure

Table 8: Replay progression on FORMALML probes.

Slice	Probe size	Exact replay	Normalized replay	Family-repair gain	Final replay
FormalML-100	10	6	6	4	10
FormalML-500	10	3	5	5	10
FormalML-all	10	3	5	5	10

Table 9: Replay-opening layers on FORMALML probes.

Layer	What is counted	100 / 500 / all probes	Interpretation
Exact replay	Raw replay under pinned provisioning	6 / 3 / 3	Baseline replay boundary on audited rows
Serializer normalization	Removes serializer noise such as <code>to_theorem</code> wrappers	6 / 5 / 5	Diagnostic normalization layer, not raw replay
Surface normalization	Joined-line and notation cleanup	0 / 0 / 0 additional	Negative control: residual failures are not superficial formatting only
Family repair	Source-aligned repair families for Bennett and ProximalGradient rows	4 / 5 / 5 additional	Diagnostic replay-opening layer, not held-out automation



Figure 6: FORMALML replay progression.

C Reproducibility and artifact summary

The supplementary material contains the LEAN 4 implementation of SUBGOALCALC, experiment runners, and curated benchmark slices for the downstream studies. The artifact is organized by run family: exact certified suites, downstream integrations, replay-backed reuse studies, and corpus-scale analyses.

The main external assets are LEAN 4 [2], AESOP [3], HILBERT [6], FORMALML [7], and MATHLIBLEMMA [4]. The supplementary material records the versions used in the experiments and describes the scope of the anonymized review snapshot. For the FORMALML replay study, the artifact separates exact extraction, environment provisioning, serializer normalization, and source-aligned family repair, making the replay boundary auditable.

The reported results consist of deterministic structural counts, exact replay counts, controller-contract outcomes, and benchmark totals. For the curated downstream slices, the benchmark definitions are included in the submitted artifact. For the corpus-scale FORMALML study, the artifact includes decomposition summaries, candidate counts, replay-progression tables, and the probe-level records used in the manuscript.

423 **Compute resources.** All reported runs are evaluation runs over certified decomposition operators,
424 curated theorem-proving slices, and corpus-processing pipelines. The experiments require no training
425 of large language models. The heaviest components are the corpus-scale FORMALML extraction
426 and replay studies and the downstream HILBERT integrations, all scripted in the supplementary
427 material.

428 **Asset licenses.** The supplementary material credits the external projects and datasets used in the
429 experiments and records their versions and applicable terms. The project-specific code and benchmark
430 slices introduced by this work are documented in the anonymized review artifact.

431 **D Broader impacts**

432 This work improves the reliability and auditability of decomposition-based theorem proving. Certified
433 decompositions expose residual obligations, dependency structure, and reconstruction evidence in
434 a form that can be checked, transformed, scheduled, replayed, and reused inside LEAN 4. These
435 properties support formal mathematics, proof engineering, and verification workflows for software
436 and hardware systems that depend on machine-checkable proof artifacts.

437 The same capabilities can strengthen formal reasoning pipelines used in high-assurance systems
438 across many domains. The artifact is a research system centered on certified proof objects, replay
439 checks, and explicit audit trails. Its main safety property is inspectability: decompositions, lifted
440 helper lemmas, schedule decisions, and replay outcomes remain tied to LEAN 4-checked evidence.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: Sections 1 and 5 state the paper’s claims, and the reported theory and experiments are aligned with those claims.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Section 7 discusses the main current boundary of the work, especially fragment selectivity for cross-corpus reuse and the benchmark scope of the dependency-sensitive controller.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: The theoretical statements appear in Section 2 and Appendix A, where the assumptions and complete proofs for the main structural results are provided.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.

- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Sections 4–5 and Appendices B–C describe the benchmarks, evaluation protocol, artifact structure, and the exact measurements required to reproduce the reported results.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: The paper is accompanied by an anonymized confidential artifact for peer review and reproducibility assessment.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.

551 • While we encourage the release of code and data, we understand that this might not be possible,
552 so [No] is an acceptable answer. Papers cannot be rejected simply for not including code,
553 unless this is central to the contribution (e.g., for a new open-source benchmark).

554 • The instructions should contain the exact command and environment needed to run to reproduce
555 the results. See the NeurIPS code and data submission guidelines ([https://neurips.cc/
556 public/guides/CodeSubmissionPolicy](https://neurips.cc/public/guides/CodeSubmissionPolicy)) for more details.

557 • The authors should provide instructions on data access and preparation, including how to
558 access the raw data, preprocessed data, intermediate data, and generated data, etc.

559 • The authors should provide scripts to reproduce all experimental results for the new proposed
560 method and baselines. If only a subset of experiments are reproducible, they should state which
561 ones are omitted from the script and why.

562 • At submission time, to preserve anonymity, the authors should release anonymized versions (if
563 applicable).

564 • Providing as much information as possible in supplemental material (appended to the paper) is
565 recommended, but including URLs to data and code is permitted.

566 **6. Experimental setting/details**

567 Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters,
568 how they were chosen, type of optimizer) necessary to understand the results?

569 Answer: [Yes]

570 Justification: Section 4 states the benchmark design and evaluation roles, while Appendix B
571 provides additional detail about suites, probes, and transfer settings.

572 Guidelines:

573 • The answer [N/A] means that the paper does not include experiments.

574 • The experimental setting should be presented in the core of the paper to a level of detail that is
575 necessary to appreciate the results and make sense of them.

576 • The full details can be provided either with the code, in appendix, or as supplemental material.

577 **7. Experiment statistical significance**

578 Question: Does the paper report error bars suitably and correctly defined or other appropriate
579 information about the statistical significance of the experiments?

580 Answer: [No]

581 Justification: Most headline results are deterministic counts over fixed certified suites and curated
582 benchmark slices rather than repeated stochastic training runs. We therefore report exact totals
583 and deltas instead of error bars.

584 Guidelines:

585 • The answer [N/A] means that the paper does not include experiments.

586 • The authors should answer [Yes] if the results are accompanied by error bars, confidence
587 intervals, or statistical significance tests, at least for the experiments that support the main
588 claims of the paper.

589 • The factors of variability that the error bars are capturing should be clearly stated (for example,
590 train/test split, initialization, random drawing of some parameter, or overall run with given
591 experimental conditions).

592 • The method for calculating the error bars should be explained (closed form formula, call to a
593 library function, bootstrap, etc.)

594 • The assumptions made should be given (e.g., Normally distributed errors).

595 • It should be clear whether the error bar is the standard deviation or the standard error of the
596 mean.

597 • It is OK to report 1-sigma error bars, but one should state it. The authors should preferably
598 report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of
599 errors is not verified.

600 • For asymmetric distributions, the authors should be careful not to show in tables or figures
601 symmetric error bars that would yield results that are out of range (e.g., negative error rates).

602 • If error bars are reported in tables or plots, the authors should explain in the text how they were
603 calculated and reference the corresponding figures or tables in the text.

604 **8. Experiments compute resources**

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Appendix C states that the work is an evaluation and systems study rather than a new large-model training effort, and identifies the corpus-scale and downstream runs as the main compute consumers.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The research is a systems and evaluation study over formal reasoning artifacts.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: Appendix C discusses both positive impacts, such as more reliable and auditable theorem proving, and negative impacts, such as strengthening verification workflows for harmful systems.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

661 **11. Safeguards**

662 Question: Does the paper describe safeguards that have been put in place for responsible release

663 of data or models that have a high risk for misuse (e.g., pre-trained language models, image

664 generators, or scraped datasets)?

665 Answer: [N/A]

666 Justification: The paper does not release a frontier generative model or a scraped dataset with a

667 high misuse profile.

668 Guidelines:

- 669 • The answer [N/A] means that the paper poses no such risks.
- 670 • Released models that have a high risk for misuse or dual-use should be released with necessary
- 671 safeguards to allow for controlled use of the model, for example by requiring that users adhere
- 672 to usage guidelines or restrictions to access the model or implementing safety filters.
- 673 • Datasets that have been scraped from the Internet could pose safety risks. The authors should
- 674 describe how they avoided releasing unsafe images.
- 675 • We recognize that providing effective safeguards is challenging, and many papers do not
- 676 require this, but we encourage authors to take this into account and make a best faith effort.

677 **12. Licenses for existing assets**

678 Question: Are the creators or original owners of assets (e.g., code, data, models), used in the

679 paper, properly credited and are the license and terms of use explicitly mentioned and properly

680 respected?

681 Answer: [Yes]

682 Justification: Existing external assets, including Lean 4, Aesop, HILBERT, FormalML, and

683 MathlibLemma, are cited in the paper, and Appendix C notes that the supplementary material

684 records the versions and applicable terms used.

685 Guidelines:

- 686 • The answer [N/A] means that the paper does not use existing assets.
- 687 • The authors should cite the original paper that produced the code package or dataset.
- 688 • The authors should state which version of the asset is used and, if possible, include a URL.
- 689 • The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- 690 • For scraped data from a particular source (e.g., website), the copyright and terms of service of
- 691 that source should be provided.
- 692 • If assets are released, the license, copyright information, and terms of use in the package should
- 693 be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for
- 694 some datasets. Their licensing guide can help determine the license of a dataset.
- 695 • For existing datasets that are re-packaged, both the original license and the license of the
- 696 derived asset (if it has changed) should be provided.
- 697 • If this information is not available online, the authors are encouraged to reach out to the asset's
- 698 creators.

699 **13. New assets**

700 Question: Are new assets introduced in the paper well documented and is the documentation

701 provided alongside the assets?

702 Answer: [Yes]

703 Justification: The paper introduces new code, certified suites, and curated benchmark slices, and

704 the supplementary material documents their role and organization.

705 Guidelines:

- 706 • The answer [N/A] means that the paper does not release new assets.
- 707 • Researchers should communicate the details of the dataset/code/model as part of their sub-
- 708 missions via structured templates. This includes details about training, license, limitations,
- 709 etc.
- 710 • The paper should discuss whether and how consent was obtained from people whose asset is
- 711 used.
- 712 • At submission time, remember to anonymize your assets (if applicable). You can either create
- 713 an anonymized URL or include an anonymized zip file.

714 **14. Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The work does not involve crowdsourcing or research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The work does not involve crowdsourcing or research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: LLMs are part of one experimental regime through the integration with HILBERT and through the decomposition-based theorem-proving literature discussed in Sections 1, 3, and 6.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.